

Call, Apply, and Bind

What is `this` in JavaScript?

In JavaScript, `this` refers to the context in which a function is called. By default:

- In a regular function call, `this` refers to the global object (window in browsers)
- In an object method, `this` refers to the object that owns the method
- In arrow functions, `this` is inherited from the enclosing scope

The Problem We're Solving

```
const user = {
  username: "Avinash",
  desg: "SDE",
  salary: "8 lpa",
  city: "Noida"
}

function details() {
  console.log("Details: ", this); // This will print 'window' object
}

details(); // 'this' = window object
```

Problem: We want `this` inside the `details` function to refer to our `user` object, not the global window.

Solution: Use `call`, `apply`, or `bind` to explicitly set what `this` should refer to.

1. CALL Method

Syntax

```
functionName.call(thisArg, arg1, arg2, arg3, ...)
```

Key Points

- **Immediate execution:** The function runs immediately
- **Unlimited arguments:** Can pass any number of arguments after the first one.
- **First argument:** Must be the object you want `this` to refer to
- **Purpose:** Changes the context (`this`) of a function call

Example 1: Basic Usage

```
const user = {
  username: "Avinash",
  desg: "SDE",
  salary: "8 lpa",
  city: "Noida"
}

function details() {
  console.log("Details: ", this);
}

details();           // 'this' = window
details.call(user); // 'this' = user object
```

Example 2: Nested Functions Problem & Solution

```
const u1 = {
  username: "Ravi",
  designation: "Frontend",

  outer: function outer() {
    console.log(this); // u1 object ✓

    function inner() {
      console.log(this); // window object ✗
    }

    inner();           // 'this' = window
    inner.call(this); // 'this' = u1 object ✓
  }
}

u1.outer();
```

Example 3: Arrow Functions (Different Behavior)

```
const u1 = {
  username: "Avinash",
  designation: "Backend",

  outer: function outer() {
    console.log("outer: ", this); // u1 object

    const inner = () => {
      console.log("Inner: ", this); // u1 object (inherited)
    }

    return inner;
  }
}

const res = u1.outer();
res(); // Arrow function inherits 'this' from outer scope
```

Example 4: Regular Function Return Problem

```
const u1 = {
  username: "Avinash",
  designation: "Backend",

  outer: function outer() {
    console.log("outer: ", this); // u1 object

    function inner() {
      console.log("Inner: ", this); // window object ✗
    }

    return inner;
  }
}

const res = u1.outer();
res(); // 'this' = window (not what we want!)

// Solution: Use call when invoking
res.call(u1); // 'this' = u1 object ✓
```

2. APPLY Method

Syntax

```
functionName.apply(thisArg, [arg1, arg2, arg3, ...])
```

Key Points

- **Immediate execution:** The function runs immediately
- **Array of arguments:** Takes exactly 2 parameters - object and array of arguments
- **First argument:** The object you want `this` to refer to
- **Second argument:** An array containing all function arguments

Example

```
const user = {
  username: "Avinash",
  designation: "Backend"
}

function details(company, city, experience) {
  console.log("Details this: ", this);
  console.log("Username: ", this.username);
  console.log("Designation: ", this.designation);
  console.log("Company: ", company);
  console.log("City: ", city);
  console.log("Experience: ", experience);
}

// Using apply
details.apply(user, ["TCS", "Noida", "2 years"]);

// Equivalent using call
details.call(user, "TCS", "Noida", "2 years");
```

When to Use Apply

- When you have arguments in an array format
- When you don't know the number of arguments beforehand
- Working with mathematical functions like `Math.max()` or `Math.min()`

```
const numbers = [1, 5, 3, 9, 2];
const max = Math.max.apply(null, numbers); // 9
// Modern alternative: Math.max(...numbers)
```

3. BIND Method

Syntax

```
const newFunction = functionName.bind(thisArg, arg1, arg2, ...)
```

Key Points

- **No immediate execution:** Returns a new function, doesn't execute immediately
- **Permanent binding:** Creates a new function with `this` permanently bound
- **Partial application:** Can pre-set some arguments
- **Reusable:** The bound function can be called multiple times
- It will not call the function immediately.
- It returns a new function in which 'this' keyword is pointing to the object references we have passed.
- To execute the function we need function references and parenthesis.

Example 1: Basic Binding

```
const user = {
  username: "Avinash",
  desg: "SDE"
}

function details(company, city) {
  console.log(this);
  console.log("Company:", company);
  console.log("City:", city);
}

const boundFunction = details.bind(user, "TCS", "Noida");
// boundFunction(); // Executes with user as 'this'

export default boundFunction;
```

Example 2: Partial Application

```
const user = {
  username: "Avinash",
  desg: "SDE"
}

function details(company, city, salary) {
  console.log(`${this.username} works at ${company} in ${city} with salary ${salary}`);
}

// Bind with partial arguments
const userAtTCS = details.bind(user, "TCS");
userAtTCS("Noida", "10 LPA"); // Only need to pass remaining arguments

// Bind with all arguments
const fullDetails = details.bind(user, "TCS", "Noida", "10 LPA");
fullDetails(); // No additional arguments needed
```

Example 3: Event Handlers

```
const button = {
  name: "Submit Button",

  handleClick: function() {
    console.log(`${this.name} was clicked`);
  }
}

// Without bind - 'this' would refer to the DOM element
document.getElementById('btn').addEventListener('click', button.handleClick.bind(button));
```

Comparison Table

Method	Execution	Arguments	Use Case
call	Immediate	Individual parameters	Quick one-time function call with specific context
apply	Immediate	Array of parameters	When arguments are in array format
bind	Returns new function	Individual parameters	When you need to reuse the function later or for event handlers

Quick Examples for Comparison

```
const person = { name: "John" };

function greet(greeting, punctuation) {
  console.log(`${greeting}, ${this.name}${punctuation}`);
}

// CALL - immediate execution
greet.call(person, "Hello", "!");           // Output: Hello, John!

// APPLY - immediate execution with array
greet.apply(person, ["Hi", "."]);           // Output: Hi, John.

// BIND - returns new function
const boundGreet = greet.bind(person, "Hey");
boundGreet("!!!");                          // Output: Hey, John!!!
```

Memory Tips

- **CALL**: Call with Commas (individual arguments)
- **APPLY**: Apply with Array (array of arguments)
- **BIND**: Bind and use later (returns a **B**ound function)

Common Use Cases

1. Borrowing Methods

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];

// Borrow push method
Array.prototype.push.apply(array1, array2);
console.log(array1); // [1, 2, 3, 4, 5, 6]
```

2. Function Currying with Bind

```
function multiply(a, b) {
  return a * b;
}

const multiplyByTwo = multiply.bind(null, 2);
console.log(multiplyByTwo(5)); // 10
```

3. Fixing 'this' in Callbacks

```
const obj = {
  name: "MyObject",
  method: function() {
    console.log(this.name);
  }
}

// Wrong way
setTimeout(obj.method, 1000); // 'this' will be window

// Right way
setTimeout(obj.method.bind(obj), 1000); // 'this' will be obj
```

Summary

- Use **call** when you want to immediately execute a function with a specific `this` context and individual arguments
- Use **apply** when you want to immediately execute a function with a specific `this` context and arguments in array format
- Use **bind** when you want to create a reusable function with a permanently bound `this` context

All three methods are powerful tools for controlling the execution context in JavaScript and solving the common problem of `this` referring to unexpected objects.